· SQL injection attacks can be avoided by using prepared statements and parameterized queries.
· Database users should be configured to have the minimum privileges they need to perform their tasks. If the API is ever compromised, this limits the damage that can be done.
· Inputs should be validated before use to ensure they match expectations. Regular expressions are a useful tool for input validation, but you should avoid ReDoS attacks.
· Even if your API does not produce HTML output, you should protect web browser clients from XSS attacks by ensuring correct JSON is produced with correct headers to prevent browsers misinterpreting responses as HTML.
· Standard HTTP security headers should be applied to all responses, to ensure that attackers cannot exploit ambiguity in how browsers process results. Make sure to double-check all error responses, as these are often forgotten.

ANSWERS TO EXERCISES

1. e - Cross-Site Request Forgery (CSRF) was in the Top 10 for many years but has declined in importance due to improved defenses in web frameworks. CSRF attacks and defenses are covered in chapter 3.
2. g - Messages from John and all users' passwords will be returned from the query. This is known as an SQL injection UNION attack and shows that an attacker is not limited to retrieving data from the tables involved